華南師範大學

《人工智能导论》课程项目

课 程 项 目 报 告

项　目　题　目：基于深度强化学习的自博弈贪吃蛇 AI

所　在　学　院：计算机学院

项　目　组　长：陈琳

小　组　成　员：曹云龙

开　题　时　间：2026 年 3 月 27 日

1 项目的名称

基于深度强化学习的贪吃蛇 AI

2 项目的主题和目标

本项目以《人工智能导论》课程理论为基础，以贪吃蛇游戏为实验环境，旨在通过系统实现多种深度强化学习算法并进行横向对比分析，加深对强化学习核心原理的理解。

在理论层面，系统梳理 DQN、Double DQN、Dueling DQN 及 PPO 四种主流深度强化学习算法的核心原理，深入理解 Q-learning 迭代公式、Bellman 最优方程、经验回放机制、目标网络更新机制、Actor-Critic 架构、PPO 裁剪目标函数以及广义优势估计等关键技术的底层逻辑。在工程层面，从零搭建适配强化学习训练的贪吃蛇仿真环境，自主设计并实现四种算法的完整训练流程，掌握深度强化学习项目从环境搭建到算法落地的全流程开发能力。在分析层面，在统一实验环境下完成四种算法的训练与对比测试，从收敛速度、训练稳定性、最终性能等多个维度量化分析各算法的表现差异，并结合可视化回放工具直观观察智能体在不同训练阶段的决策行为演变。

3 项目简介

本项目从零搭建了 12×12 网格的贪吃蛇仿真环境，采用三通道图像编码状态表示（蛇身、食物、蛇头），定义上下左右四个离散动作，设计了由食物奖励、死亡惩罚、步惩罚和距离变化奖励组成的组合奖励函数。在此基础上实现了 DQN、Double DQN、Dueling DQN 和 PPO 四种深度强化学习算法，所有算法在同一训练框架下通过命令行参数统一切换。

DQN 系列（标准 DQN、Double DQN、Dueling DQN）共享经验回放与目标网络机制：经验回放池容量 50000，每次采样 128 条样本进行训练；目标网络每 300 步同步一次在线网络参数；探索率从 1.0 指数衰减至 0.01。三种变体的区别在于目标值计算方式——标准 DQN 直接使用目标网络的最大 Q 值，Double DQN 用在线网络选择动作后

再用目标网络评估，Dueling DQN 则将 Q 值分解为状态价值和动作优势两部分。PPO 基于 Actor-Critic 架构独立实现，采用三层卷积的共享 CNN 骨干网络，实现裁剪目标函数、广义优势估计（GAE）与熵正则化三项核心机制，每次收集 3 个 episode 后执行 8 轮小批量更新。

训练过程中经历多轮参数调试与优化。修复了探索率双重衰减 Bug，三轮调整奖励函数使食物净收益从负转正，优化 CNN 网络结构将特征图分辨率从 3×3 恢复至 6×6，最终确定学习率 5e-4、批大小 128、目标网络更新频率 300 步等最佳参数配置。

实验结果表明，Dueling DQN、Double DQN 和标准 DQN 在 10000 回合训练后均能达到 300 以上的平均奖励，其中 Dueling DQN 表现最优，其价值-优势分解结构在动作价值相近的场景中提供了更稳定的估计。PPO 在本场景下未实现有效收敛，分析认为 on-policy 训练模式的样本利用效率较低，且小网格离散动作空间更适配值函数方法。

此外，项目还开发了一套训练阶段演进可视化回放工具。采用录制-回放分离方案，训练时每 200 回合轻量录制逐帧游戏数据，训练后可实现多算法并排回放、训练曲线渐进延伸和 Q 值热力图显示，直观展现智能体从随机试探到策略收敛的完整学习过程。

4 整个项目研发过程中个人的主要工作和收获

在本项目中，我主要负责强化学习训练平台的搭建与调试工作，完成了四种强化学习算法的代码实现与功能开发，开展了智能模型的训练、参数调试与效果优化，设计并实现了可视化交互界面，并参与了相关技术文档的编写与核查工作。

4.1 算法实现与训练优化

在技术开发方面，我首先从零搭建了贪吃蛇仿真环境，实现了完整的训练循环。这个过程让我对强化学习的基本模式有了具体的认识——智能体观察环境状态、选择动作、获得奖励、调整策略，不断循环迭代，书上的概念在实际代码中一步步落地。

在算法实现与调优阶段，我体会到了奖励函数设计对训练效果的决定性影响。初始参数设置下（死亡惩罚-100、食物奖励+10），智能体几乎不学习；经过多轮调整，最终将奖励参数定为死亡-20、食物+30、步惩罚-0.05，并加入距离奖励机制，训练才开始收敛。调参过程中我发现奖励函数各分量的比例关系直接决定了智能体的行为倾向，其重要性不亚于算法本身。

在训练过程中，我还学习并应用了多种提升训练效果的手段：经验回放机制通过随机采样打破样本之间的时序相关性，让模型能从更多样的历史经验中学习；目标网络为 Q 值提供了稳定的优化目标，避免了单网络更新中目标不断变化的问题；探索率从高到低的衰减策略平衡了前期的探索需求与后期的利用效率。此外还修复了一个探索率双重衰减的 Bug——原代码中两处衰减逻辑叠加导致探索率骤降，修复后训练才正常启动。这个经历让我意识到代码中的细节错误可能导致训练完全偏离预期。

随着调试的深入，我逐渐认识到超参数之间存在耦合关系。例如网格尺寸缩小后模型参数量下降，可以适当提高学习率来加快收敛；学习率调高后又需要调整目标网络更新频率来维持稳定。几个参数需要整体考虑，逐一调优效率较低。

4.2 工程开发与调试习惯

在代码结构方面，我体会到合理的代码设计能让开发顺利很多。项目中使用 config.py 统一管理超参数，改参数不用到处翻代码；不同算法通过 –algo 命令行参数切换，而不是各写各的训练脚本；环境模块和智能体模块分开编写，互不干扰。这样的结构使得后续添加新算法或调整参数都非常方便。

在开发习惯上，我养成了每写一个小模块就及时测试的习惯——写完游戏环境就跑几步看看蛇移动和食物生成是否正确，写完神经网络就输入一组数据检查输出形状是否符合预期。小问题边写边发现边改，如果等到所有代码写完再统一测试，出了问题很难定位。同时，训练过程中输出的日志信息也发挥了很大作用。把每回合的奖励、损失、步数、探索率都打印出来，训练过程中就能实时判断模型在变好还是原地踏步，不用等到全部跑完才去分析问题，大大提高了调试效率。

在可视化方面，我体会到可视化手段对调试的帮助。训练曲线可以直接反映模型是否在学习，游戏画面回放能观察智能体的具体行为策略，Q 值热力图则显示了网络

的决策依据。没有这些可视化工具，仅靠数值指标很难定位训练中的问题。此外，我还意识到开发过程留痕的重要性。训练过程中的参数调整、Bug 修复、结构改动，都要及时记录下来，不然到写报告的时候全靠回忆，容易遗漏，也不好体现工作量。例如探索率衰减 Bug 的发现和修复过程，如果在改代码时顺手记录一笔，后面写文档就省事很多。

4.3 文档编写与团队协作

在文档编写方面，我也有不少收获。首先认识到文档工作很重要，是展示工作量和成果的地方，报告需要图文并茂，纯文字会让老师看得很累；每个小节下面的内容最少要有两个段落，否则分节显得没有必要；文档内容最好有前后照应，格式要规整，段落需要首行缩进，这些细节能体现态度的认真。

在写作风格上，最好不用分条例点式的表述，那样显得 AI 味过重，要注意不要让文字的表述太有 AI 痕迹，不然可能阴沟里翻船。在工具使用上，我学会了使用 Pandoc 将 Markdown 文件生成为 Word 文档，可以利用一个已有格式的 .docx 文件作为模板，这样就不需要再手动调整生成文档的格式。.doc 文件可以通过 Word 转为 .docx 文件，然后直接作为 Pandoc 的生成模板。图片和视频的生成也可以借助 AI 工具来完成，有时候比自己手绘的更美观。

在团队协作方面，我认识到两人一定要及时沟通，互相核对彼此的理解是否一致——代码是否与预期有偏差，文档是否与实际实现的代码相符，在开发、测试和训练过程中要记录好过程，这样编写文档时也更轻松准确。有问题和疑惑要及时与老师沟通，询问老师的修改意见与要求。在技术实现方面，我体会到 AI 辅助开发的价值，它可以让人从具体的代码工作中解放出来，将精力集中在程序逻辑与结构的设计上，具体的实现细节交给 AI 完成。

5 课程项目中目前存在的问题

当前模型在常规游戏场景中能够实现稳定的寻食与避障，但仍存在以下局限。在高阶决策能力方面，当蛇体较长、活动空间狭窄时，偶尔出现路径规划失误或陷入局

部最优决策的情况。例如蛇头在狭窄通道中折返、或在食物与蛇尾位置交错时选择低效路线。分析认为，单层卷积的网络结构虽然满足轻量化需求，但其特征图尺寸有限，难以同时建模蛇头、蛇尾和食物的全局空间关系，导致在极限场景下的空间推理能力不足。

在训练后期收敛方面，模型在达到 300+ 的平均奖励后性能提升趋于停滞。轻量化网络的特征提取能力可能已接近上限，继续增加训练回合数也难以带来明显的性能提升。同时在样本利用方面，长期训练后经验回放池中积累了大量相似度较高的样本，模型反复学习已经掌握的交互模式，对核心误区的针对性修正效率逐渐降低。这一问题在 off-policy 算法中较为常见——经验池中历史样本的分布与当前策略的决策分布存在偏差，导致网络更新方向不够精准。

此外，PPO 算法未能收敛。尽管完整实现了 PPO 的训练流程，但奖励曲线始终在负值区间波动。可能的原因包括超参数配置未达到最优区间、网络结构设计或训练轮次不足以支撑策略梯度方法的有效收敛等，需要对 PPO 的训练策略进行更加深入的调试。

6 项目实践过程中的自我感想

整个项目的训练过程，就像在”养宠物”一样。第一次跑训练的时候，看着训练日志里的数据一点点往外弹，心里还挺兴奋的。通过一次次调参，把奖励分数慢慢从负的变成正的，这个过程确实很有成就感。印象最深的是有一天上午在教室跑训练，到中午吃饭还没跑完，我就抱着笔记本电脑走到食堂，把电脑放在餐桌上让它继续跑，自己去打饭。那一刻感觉自己像个在照顾孩子的家长，走到哪都得带着它。

因为没有 GPU，所有的训练都在 CPU 上完成，每一步计算都相对较慢。但回过头来看，这个限制反而让我学得更扎实。如果当时直接有 GPU 可用，我可能不会去深入理解每一层网络的作用和每个参数的意义——反正跑得快，多试几次就行了。正是因为计算资源有限，每一步优化都得想清楚再动手，反而逼着自己把原理弄明白了。

写报告一直是我最头疼的事，每次对着模板的标题不知道从哪下笔。原本的分工是组长负责文档、我负责代码，但进展报告出来后，我发现对实验过程的描述和实际代码有不少出入，只能硬着头皮自己上手改。改的过程中发现有些问题自己也没完全

弄清楚，又回头去翻训练日志和代码，一边改一边把整个项目重新理解了一遍。到期末报告的时候，仍然是组长先负责，我看了初版后发现文档的核心部分和实际情况有较大出入，索性直接把文档的工作接了过来，搞完初版以后给组长审核，组长指出了格式和内容上的问题（具体问题可见 4.3），又花了不少时间调整格式、添加图表，虽然过程挺折腾，但写文档的能力也提高了

最后，关于 AI 工具的使用，我也有了一些反思。AI 擅长的是编写具体的代码片段，但对于提出让人眼前一亮的想法，它还是无能为力的。项目架构应该怎么设计、问题排查从哪个方向入手、参数应该往哪个方向调，这些还是要靠自己来判断。能写好代码不代表能做好项目，人的价值在于知道要做什么、知道往哪个方向去。